

Chapter 1

Impact of VLSI on Computer Architectures

C. V. RAMAMOORTHY
Y. W. MA*

Department of Electrical Engineering
and Computer Science
University of California, Berkeley
Berkeley, California

I. Introduction	2
A. State of the Art of VLSI Technology	2
B. Limitations on Integrated Circuit Technology	3
C. Trends in the Use of VLSI Technology in Computer Architecture	5
II. Single-Chip Computers	6
A. Motivations for Single-Chip Computers	6
B. Features and Characteristics of Single-Chip Computers	8
C. Impact of VLSI on Single-Chip Computers	9
D. Example of a VLSI Single-Chip Computer— X-Node	12
III. Special-Purpose VLSI Chips	13
A. Migration of Functions—VLSI Implementation of Algorithms	13
B. Distributed Intelligence in System Components	21
IV. Conclusion	22
References	22

* Present address: Department of Computer and Information Science, University of Pennsylvania, Philadelphia, Pennsylvania.

I. INTRODUCTION

A. State of the Art of VLSI Technology

Very large scale integration technology means upward of many hundreds of thousands of switching devices on a single silicon chip with feature size approaching 1 micron. Since the 1960s, integrated circuit (IC) technology has advanced from a few to tens of thousands of transistors on a single silicon chip. For the first 15 years since its inception, the progress in making ICs in ever more complex structure has moved in an exponential fashion, at a rate doubling the number of transistors that could be placed on a single chip (with tolerable yield) every year [1]. Over the past few years, this growth rate has slowed to doubling every 18 to 24 months. The present state of the art of IC technology is about 70K devices on a single chip, such as the Motorola MC68000 or the 64K memory chip. The past history and the future trend for IC technology are illustrated in Fig. 1.

The rapid advance of IC technology has been made possible mainly by the improvements in fabrication technology. This improved technology has reduced the density of defects, which makes larger chips feasible.

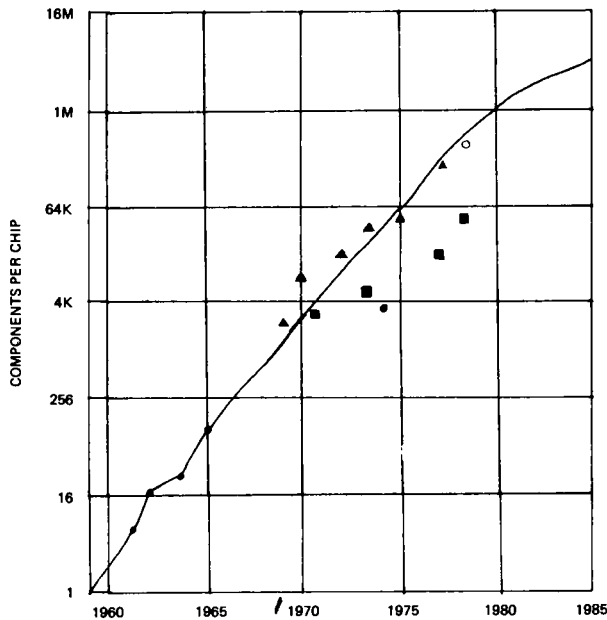


Fig. 1. The history and trend of IC technology: bipolar (●); MOS logic (■); MOS memory (▲); and bubble memory (○).

Moreover, the improvements enable the use of much smaller size transistors and wires, which increases the density of the circuitry. At the same time, the reduced size also results in faster and/or lower power operation.

B. Limitations on Integrated Circuit Technology

Although IC technology continues to advance rapidly, there are several factors that constrain the integration level of future silicon IC technology. These factors can be categorized into physical limits, technological limits, and complexity limits [2].

Physical limits include the velocity of light, the principle of uncertainty, entropy (irreversibility), and thermal energy. They determine the fundamental limits that present barriers to switching speed and power dissipation that cannot be surpassed. These physical limits suggest an ultimate density improvement thousands of times greater than that attained by present-day IC technology.

Technological limits include those of fabrication techniques, materials constants, and electrical parameters. The constraints imposed by technological limits can usually be circumvented by such methods as using different materials, lowering the operating temperatures, changing the structure, and improving the cooling techniques.

Complexity limits refer to the human inability to design circuitry that involves very large numbers of components. In other words, these are conceptual limitations. Complexity involves product definition, design engineering changes, testing, on-chip redundancy, computer-assisted design, packaging, and so on. The complexity limit directly affects the design cost of VLSI chips [1]. The amount of effort required for product definition, design, and layout in man-hours per month, starting from the first planar transistor of 1959 and projecting beyond that, is shown in Fig. 2. This design effort is also plotted on the logarithmic scale in Fig. 3. It can be seen that the effort explodes exponentially, doubling every $2\frac{1}{2}$ years. Since the cost in man-hours per month inflates at the rate of at least 10%/yr, the cost is seen to double every two years. Along with the fact that the IC density also doubles every two years, this results in a constant cost per element to define, design, and lay out complex semiconductor ICs.

When contrasted with the design cost, the manufacturing costs of ICs are mainly independent of device complexity. Whereas manufacturing has been the dominating cost in the past, with the present high density of IC, the situation is now reversing; the design cost is now becoming more important, implying that in the future considerable effort will be needed in

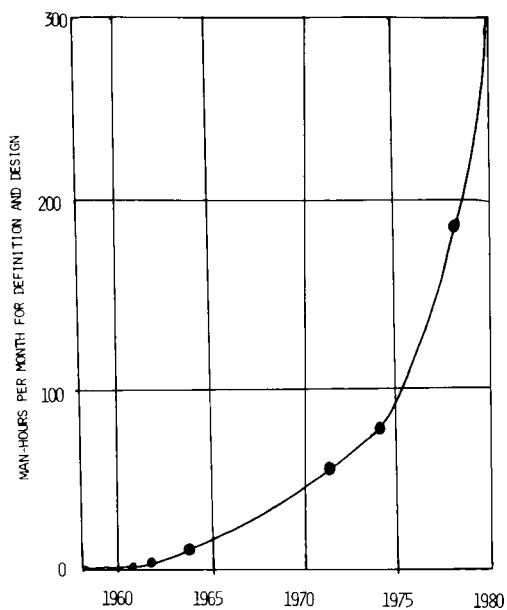


Fig. 2. The man-hours per month for IC definition, design, and layout (after Moore [3a]).

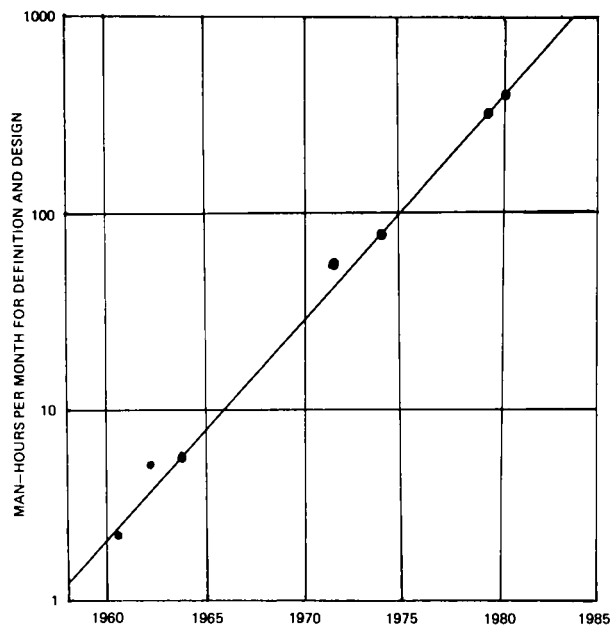


Fig. 3. The curve in Fig. 2 replotted on a logarithmic scale (after Moore [3a]).

product definition and design technology in order to surpass the limit imposed by complexity.

Practically speaking, VLSI technology is a statement about system complexity rather than transistor size or circuit performance. It stands for a technology that is capable of creating systems so complicated that the management of the complexity itself overwhelms all other difficulties. The trend to increase the functions on a single silicon chip will continue, and despite the various limiting factors that constrain the growth of IC complexity, the density of the transistor can increase at least an additional 400 times before the fundamental limit is reached. Ultimately, it will be the practical, rather than the absolute, limit that constrains the size of IC chips, and the optimal size of IC chips will be the best balance between yield and complexity.

C. Trends in the Use of VLSI Technology in Computer Architecture

With the large number of switching elements available in a single chip as promised by VLSI technology, the question that arises naturally is: What can we do with this technology and how can we best utilize it? Basically, there are two different architectural directions in which VLSI-based computers can proceed [3]. The first is essentially the traditional one, which technology has been following since the 1960s, that is, to put more and more functions on a chip as well as to make the chip run faster and faster. By integrating onto a single chip the CPU, the memory, and the input/output circuitry of an older design that previously required several chips, the single-chip computer architecture results. Since development in the direction of designing single-chip stand-alone computers will be plagued by the need for compatibility with prior designs, the architectural progress along this direction will be relatively slow.

The other architectural direction takes a fresh look at new technology and many recently emerged computer applications. By exploiting the advantages of VLSI technology—that a complex CPU or even a full single-chip microcomputer costs only a few dollars—it becomes economical to design systems using a multiplicity of microcomputers providing more processing power than would be possible or practical using a single CPU with traditional architecture. Essentially, this direction considers the interconnection of VLSI chips, which can either be general-purpose single-chip computers or special-purpose VLSI chips to form highly concurrent computer systems. This direction implies the exploration of many new architectural concepts and algorithmic designs as well as the opening up of many new computer applications.

One serious drawback of VLSI technology is the limited number of pins on each chip. While VLSI chips provide an exponentially growing number of gates, the number of pins they provide remains almost constant. It has been observed experimentally that if the number of circuits on a chip is proportional to the volume of a sphere, then the number of pins required to serve these circuits is proportional to the surface of the sphere [4]. As a result, communication becomes a very difficult design problem in the interconnection of VLSI chips. Due to the insufficient communication power and the high design cost of VLSI chips, computer systems employing VLSI technology will thus need to employ many architectural concepts that depart sharply from past and present practices [5,6].

II. SINGLE-CHIP COMPUTERS

A. Motivations for Single-Chip Computers

For conventional computer systems in which the processor and memory are implemented in separate IC chips, the communication between processor and memory remains a major and difficult design problem when a high bandwidth requirement is imposed on the system [7,8]. The difficulties are due to the following two reasons: First, because of the different loading capacities, the delay–power product of a connection residing within a single IC chip is much smaller than those interconnecting separate chips. At present, the ratio is more than two orders of magnitude and will become larger as proper scaling of MOS circuits leads to faster and smaller circuits, which operate at lower power levels. Although it is possible to drive the signals going through the package pins as fast as minimum-sized gates can drive internal signal paths by building large power-consuming driver circuits onto the chip, at present external signals speeds are still about an order of magnitude slower due to practical trade-offs. Hence, bringing a signal from one chip to another results in a significant performance penalty, either as increased power consumption or as excessive delay. Therefore, a high memory bandwidth is very difficult to obtain in conventional computers.

The second factor that limits the memory bandwidth is the limited number of package pins for standard semiconductor memory. As shown in Fig. 4, a standard semiconductor memory is usually designed as a square matrix, since the chip area is minimized when the X and Y decodes are of the same size. The X decode is used to select one of the words, while the Y decode is used to select one of the bits in that particular word.

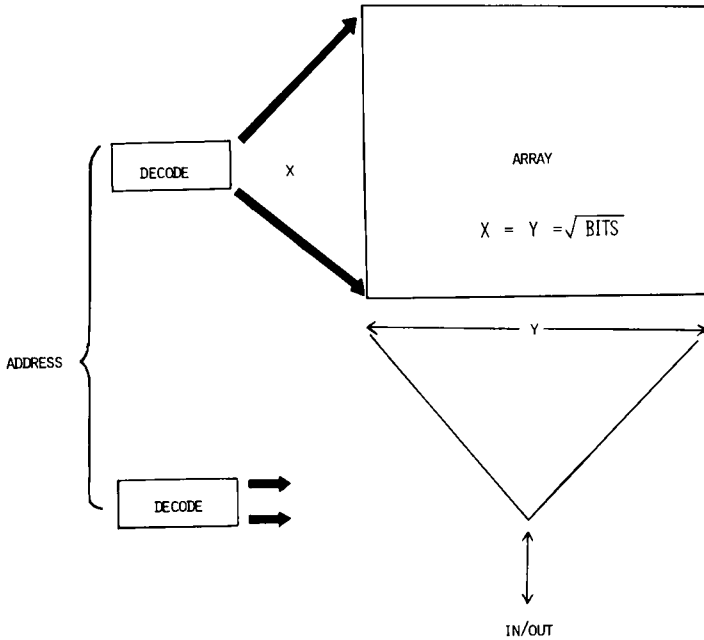


Fig. 4. Standard memory device architecture.

In order to minimize the number of package pins, a “by one” organization is usually employed, which leads to severe waste of the available memory bandwidth. For example, in a 64K-bit memory device, which has $X = Y = 256$, the “by one” organization has 255/256, or 99.6%, of the available memory bandwidth unused.

In order to overcome these difficulties, techniques such as interleaved memory and fetch of multiple computer words each cycle have to be used when the required bandwidth exceeds that of a single memory module. As a result, providing high memory bandwidth usually results in a costly design. A new approach to achieve high bandwidth without incurring high cost is to integrate the processor and memory onto a single IC chip. This approach is made feasible by the increasing number of transistors on a single silicon chip. Since the data path between the processor and the memory then resides within a single chip, the constraints imposed on memory bandwidth in conventional computers disappear automatically. Another advantage of single-chip computers is that the reduced number of chips and connections in the computer system also results in higher reliability. Hence, the design and development of self-contained, single-chip computer systems with processor, memory, and input/output logic all con-

tained on a single chip of silicon are inevitable and necessary with the emergence of VLSI technology.

B. Features and Characteristics of Single-Chip Computers

1. Design Considerations for Single-Chip Computers

For a given typical NMOS process used in the manufacturing of single-chip computer, the manufacturing cost can be expressed as

$$\text{manufacturing cost} = K(10^{0.0243A}),$$

where A is the area of chip in square mils per 1000 and K a constant for a given process.

Because of this exponential relationship between manufacturing cost and chip area, any small decreases in chip area can lead to large decreases in manufacturing cost and vice versa. For general-purpose single-chip computers that are produced in large quantities, a small reduction in the manufacturing cost of the single chip will result in a large savings in manufacturing all the required quantities. For this reason, one of the major design principles of single-chip computers is to achieve minimum chip area [8].

The area of a single-chip computer can be divided into three regions: processor, memory, and buffers and bonding pads, as shown in Fig. 5. The geometry of the buffers and pads can be thought of a picture frame surrounding the CPU and memory that is about 15 to 20 mils wide. Hence, the approximate area of a single chip is

$$\text{area} = [(CPU + \text{memory})^{1/2} + 40]^2.$$

Because the memory area is fixed for any selected technology and quantity of memory, the CPU area is thus the only variable parameter for any design. Consequently, the CPU area has a significant influence on the

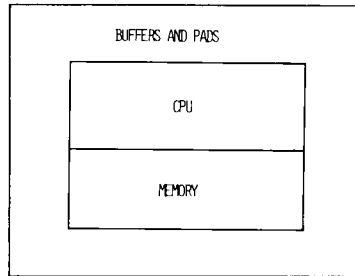


Fig. 5. Single-chip computer area partition.

resulting area and cost, and any design with smaller CPU area will be preferable to the designer.

The minimization of chip area also affects the design trade-offs of the on-chip memory. While the bandwidth constraint on memory is relaxed for single-chip computers, the design trade-off for memory bandwidth is complicated by the bus that provides the communications between the memory section and the other sections of the single-chip computer. As the bus widens, it is more difficult to route and, hence, more chip area is used. In order to save chip area, the number of bits per word has to be minimized. To date, the best compromise is achieved by memory and bus widths of 8 and 16 bits. For recent products, such as Intel 8089 [9], a 20-bit register and internal bus is used.

2. Architecture for Single-Chip Computers

There are two basic types of architecture for single-chip computers, namely, the *Harvard architecture* and the *von Neumann architecture*. The Harvard architecture is characterized by disjoint instruction and data memories, whereas in von Neumann architecture, both instruction and data reside in the same memory. The advantage of the Harvard architecture is that it provides great flexibility in word-size selection because it is not necessary to have the length of the instruction word as well as the various registers equal to or a factor of the data word size. In addition, the separation of the instruction and data memories allows the overlapping of instruction and data access. For any given technology, this increased bandwidth results in higher performance over the single-chip computer based on the single-memory von Neumann architecture. The advantage of the von Neumann architecture is that, because data and instruction are intermixed in one address space, instruction can be used as data and vice versa. To date, the Harvard architecture still provides the basic architecture for many single-chip computers, but single-chip computers based on the von Neumann architecture have started to emerge. For example, the TMS9940 [10] is based on the von Neumann architecture. For the future, with the single-chip computer used more as a general building block for highly concurrent systems than stand-alone computers, the von Neumann architecture will become more important.

C. Impact of VLSI on Single-Chip Computers

For the past few years, self-contained, single-chip computers have been used mostly in stand-alone control systems. With the high design cost of VLSI chips, the products have to be carefully designed to be of general

usefulness so that large quantities will be needed and thus a reasonable price can be achieved. In addition, since the inherent maximum speed that can be obtained from silicon integrated circuits is limited by the smallest feasible dimensions in the active constants as well as by materials constants, to obtain more computational power from a given technology, the complexity of the system has to be increased so that more computational steps can be performed simultaneously. Due to the high design cost of VLSI chip, sophisticated computer architecture is unsuitable for VLSI single-chip computers.

One feasible approach to achieving high computational power using VLSI technology is to employ the multiprocessor architecture. This approach has the advantage that only a small number of building blocks have to be designed. Furthermore, the logic of these building blocks is relatively simple. Hence, the design of general-purpose single-chip computers that can be used as building blocks for multiprocessor systems will become an important area for VLSI single-chip computer research.

With the large number of transistors provided in a single silicon chip, many architectural features available only in large computers or minicomputers can be provided in single-chip computers. Features such as large local memory, support for complex data structures, flexible and powerful instruction set, and interprocessor communication can all be provided by the single-chip computer [7].

1. Large On-Chip Memory

It has become evident that large local memory for program and data is needed in single-chip computers as increasing address spaces are provided by small computers. For example, the Intel 8086 has a 1024K-bytes memory, the Z8000 a 8192K-bytes memory, and the MC68000 a 16384K-bytes memory. An attractive approach proposed in the design of X-node and P1985 to provide on-chip memory with high density and fast access time is to implement a memory hierarchy combined of dynamic and static RAMs [7, 11, 13]. Dynamic RAM provides high-density memory but its access time is relatively slow. On the other hand, static RAM provides faster access times but lower density. By using the static memory as a small cache and the dynamic RAM as a large main memory, an on-chip memory with fast access time and high density can then be achieved.

2. Support of Complex Data Types and Operations

While the traditional 8-bit microprocessor only supports simple operations on 8- and 16-bit integers, the more recent designs support a much wider variety of data types and operations. For example, the MC68000

supports a large number of data types, including integer, multiprecision integer, logical, Boolean, decimal, character, address, floating point, and string [11]. Different operations are provided to support these data types. For future single-chip computers, complex data structures such as arrays and structures can also be supported. The associated operations, such as bound checking and address calculation of specific elements, may also become cost-effective when implemented by the appropriate hardware.

3. Support of High-Level Language

The importance of high-level language support has become more evident as an increasing percentage of programs developed for microprocessors are written in high-level languages such as Fortran and Pascal. The advantage of using high-level language is that it can reduce the cost of coding, debugging, and maintenance. The MC68000 supports high-level languages at both compilation time and execution time, with a clean, consistent instruction set, hardware implementation of commonly used functions, such as address calculation, multiply and divide, and a set of special-purpose instructions to manipulate the run-time environment of a high-level language program. The language constructs that are facilitated by these special-purpose instructions include array accessing, limited-precision arithmetic, looping, Boolean expression evaluation, and procedure calls. The future single-chip computer can have the capability of supporting a large variety of high-level languages.

4. Support for Communication

In order to arrive at a single-chip computer suitable for use as the building block for a large multiprocessor system, the communication problem that has not yet been addressed properly in current single-chip computers has to be tackled appropriately. All the necessary switching circuitry and communications hardware have to be part of the design in the single silicon chip.

5. Extensive Use of Microprogramming

Because of the high complexity of VLSI chips, the lack of suitable design tools, and the difficulties in testing these chips because of the large number of transistors and small number of pins, it is necessary to have the simplest and most regular designs for VLSI-based single-chip computers. The most complicated part in a processor is the control circuitry. Microprogramming, which leads to a more structured and regular design of control logic as compared to the traditional approach to implementation con-

trol based on carefully minimized logic functions using gates that are placed irregularly in the available space between other circuit modules, is very attractive for the implementation of control in VLSI processors.

In addition, microprogramming is also essential for testing complicated VLSI designs. Since the ratio of the number of internal nodes to the number of accessible points in integrated circuits is increasing exponentially, it is impossible to test VLSI circuits by applying test patterns from outside. Hence, the use of microprogramming to enable "microdiagnostics" is very crucial for VLSI design. A further advantage of microprogramming is its flexibility and adaptability. For example, microprogramming with writable control storage can support different high-level languages by switching to different instruction sets.

D. Example of a VLSI Single-Chip Computer—X-Node

The X-node is a single-chip computer based on VLSI technology and is intended to be used as the fundamental building block for a tightly coupled, hierarchical, tree-structured multiprocessor system called *X-tree* [12–14]. The interconnection topology of the X-tree is a binary tree enhanced with additional links to form a half-ring or a full-ring tree. In order to allow the system to expand to arbitrary size, unbounded variable-length addresses are employed throughout. Communication in the tree is based on messages. All input or output and secondary memory are at the frontier of the tree.

Because of the high bandwidth imposed by the architecture of the X-tree, it is important that the processor not be involved in the menial task of routing messages through X-tree. Hence, computation must occur in parallel with communication. In the X-node, a self-controlled switching network with its own I/O buffers and controllers is included. The most important part of the switching network is the time-multiplexed bus. The associated parasitic capacitances are rather low since the bus resides within one chip. Hence, for a given amount of drive power, the resulting bandwidth is about an order of magnitude higher than that through the package pins associated with the I/O ports. As a result, the attached six to eight ports can be served at their full capacity.

Memory contained in the X-node only acts as a cache to the secondary memory contained at the leaves of the X-tree. A large memory space is designed in order to alleviate the paging problem. The on-chip memory hierarchy, consisting of a high-density RAM and three high-speed static caches dedicated to data, instructions, and microcode, is used to provide a large memory with fast access time.

The X-node exemplifies the off-the-shelf, general-purpose single-chip

computer that can be used as a modular building block for the construction of a computing system of any size. The architecture is based entirely on VLSI technology with about 100,000 gates and 500,000 bits of storage. Although at present the designs of this type of single-chip computer are rather limited, we foresee that active research will be carried out in this area in the near future.

III. SPECIAL-PURPOSE VLSI CHIPS

The decreasing cost of hardware has invoked the following trends in the design of special-purpose hardware modules: (1) the migration of functions from software to hardware modules in order to obtain high execution speed and (2) the distribution of intelligence into different computer systems' modules, such as memory and I/O processor, in order to obtain better balance of system performance by removing bottlenecks in the system. Because special-purpose VLSI chips are usually produced in relatively small quantities, the design cost must be kept low in order to make it cost-effective.

A. Migration of Functions—VLSI Implementation of Algorithms

The hardware implementation of an algorithm can be executed faster than the software implementation because of the inherent fast execution speed of hardware. Furthermore, by using more hardware components, additional speedup can be obtained. For example, using a linearly connected network of size $O(n)$, both the convolution of two n vectors and the n -point discrete Fourier transform can be computed in $O(n)$ units of time, rather than $O(n \log n)$, as required by the sequential fast Fourier transform algorithm [15]. The design cost of VLSI chips to implement algorithms can be reduced drastically if the algorithm is designed carefully in the first place. The complexity of designing special-purpose chips is about the same as that of designing a high-level algorithm for the same problem if the underlying algorithm is "good" and if a proper methodology to transform the good algorithm into a final layout in a more or less mechanical way is used. Hence, for a class of problems that can be solved by good algorithms, the design of special-purpose VLSI chips to implement them becomes feasible and cost-effective.

1. Properties of Good VLSI Algorithms

Algorithms that perform well on conventional random access computers are not always the best for VLSI implementation. Since VLSI has

high computational power but limited communication capability, good algorithms for VLSI implementation are not necessarily those with minimal computation. Hence, the characteristics of a good algorithm in the context of VLSI depart from those for conventional random access computers. On the other hand, the following three properties exemplify a good VLSI algorithm [16, 17].

- (i) The implementation of the algorithm requires only a few different types of simple cells.
- (ii) The data and control flow of the algorithm are simple and regular. This property ensures that the cells can be connected by a local and regular interconnection to form a network, in order to minimize long-distance or irregular communication.
- (iii) The algorithm uses extensive pipelining and multiprocessing.

Algorithms with these properties are called *systolic* algorithms. The design cost of special-purpose VLSI chips for these algorithms can be kept low for the following reasons:

- (i) Since most of the cells on the chip are copies of a few basic ones, only a few different, simple cells need to be designed and tested.
- (ii) Regular interconnection ensures that the design is modular and extendable. Hence, a large chip can be designed by combining the designs of small chips.
- (iii) The pipelining and multiprocessing property enables the performance requirement of a special-purpose chip to be met by including many identical cells on the chip.

The design cost of a special-purpose VLSI chip can be greatly reduced if a good algorithm is used. Hence, the most important step in the design of these chips lies in the design of the underlying algorithm. Because special-purpose chips are usually produced in small quantities, low-level optimization at the circuit or layout design level leading to minor improvements in the overall performance but drastic increases in design time may not be worthwhile.

2. Classification of VLSI Algorithms

Many systolic algorithms have been designed recently. These algorithms can be classified according to the underlying geometry as



Fig. 6. One-dimensional array geometry.

follows: one-dimensional linear arrays, two-dimensional square arrays, trees, and shuffle-exchange networks, as shown in Figs. 6–10 [18]. Some of the VLSI algorithms are listed in Table I according to this classification. The characteristics of each geometry are given in the following:

(i) One-dimensional linear arrays. This is the simplest way of connecting cells and provides the basis for other communication geometries. The main characteristic of the linear-array geometry is that it can be viewed as a pipe. Hence, it is natural for pipelined computations.

(ii) Two-dimensional arrays. This geometry is natural for problems involving matrices, such as graph problems defined in terms of adjacency

TABLE I
Examples of VLSI Algorithms^a

Communication geometry	Example
One-dimensional linear arrays	Matrix–vector multiplication FIR filter Convolution DFT Carry pipelining Pipeline arithmetic units Real-time recurrence evaluation Solution of triangular linear systems Constant-time priority queue, on-line sort Cartesian product Odd–even transposition sort
Two-dimensional square arrays	Dynamic programming for optimal Parenthesization Numerical relaxation for PDE MERGE sort FFT Graph algorithms using adjacency Matrices
Two-dimensional hexagonal arrays	Matrix multiplication Transitive closure LU-decomposition by Gaussian Elimination without pivoting
Trees	Searching algorithms Queries on nearest neighbor, rank, etc. NP-complete problems Systolic search tree Parallel function evaluation
Shuffle-exchange networks	FFT Bitonic sort

^a From Kung [78].

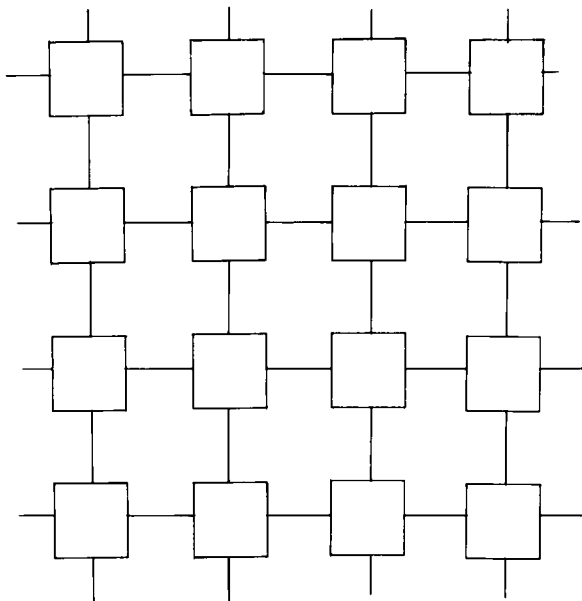


Fig. 7. Two-dimensional array geometry.

matrices and numerical solutions to discretized partial differential equations.

(iii) **Hexagonal arrays.** This structure has the property of symmetry in three directions. Hence, after a binary operation is executed at a cell, the result and the two inputs can all be sent to the neighboring cell in a completely symmetric way. This type of computation eliminates a possible separate loading or unloading phase, which is always needed in algorithms using two-dimensional arrays.

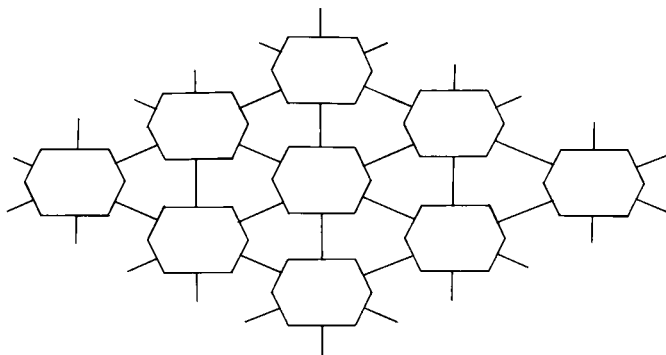


Fig. 8. A 3×3 hexagonal array.

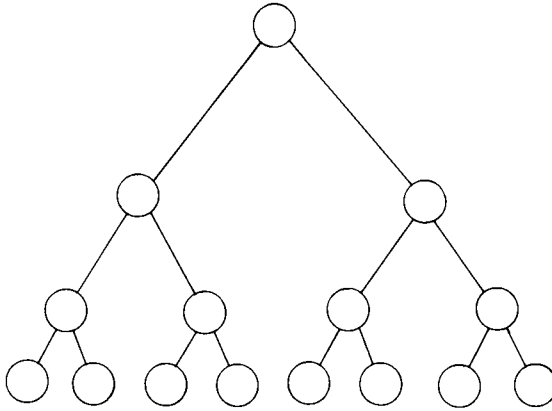


Fig. 9. A tree geometry.

(iv) Trees. The advantage of tree structure is that the broadcasting, searching, and fan-in operations can all be executed in logarithmic time. However, cells at high levels of the tree can easily become bottlenecks if the majority of communications are not confined to cells at low levels.

(v) Shuffle-exchange networks. A shuffle-exchange network can be defined as follows: Let m be an integer and let the network have $n = 2^m$ nodes. Assuming that the nodes are named $0, 1, \dots, s^m - 1$, let $i_m i_{m-1} \dots i_1$ denote the binary representation of any integer i , for $0 \leq i \leq 2^m - 1$. The shuffle function can then be defined by

$$S(i_m i_{m-1} \dots i_1) = i_{m-1} i_{m-2} \dots i_1 i_m,$$

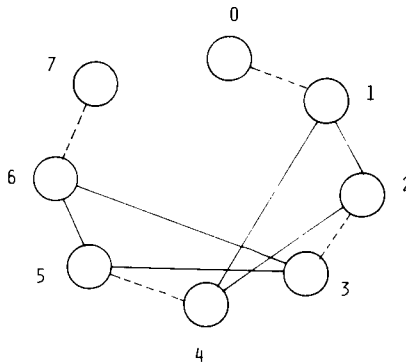


Fig. 10. A shuffle-exchange network, where the solid lines denote the shuffle and the dashed lines the exchange.

and the exchange function can be defined by

$$E(i_m i_{m-1} \cdots i_1) = i_m i_{m-1} \cdots i_2 \bar{i}_1.$$

The network is called a *shuffle-exchange network* if node i is connected to node $S(i)$ for all i and to node $E(i)$ for all even i . By using the exchange and shuffle connections alternately, data at pairs of nodes with names differing by 2^i can be brought together for all $i = 0, 1, \dots, m-1$. This type of communication structure is common to many algorithms. For example, the n -point fast Fourier transform can be done in $O(\log n)$ steps on the network if the processing elements can perform addition and multiplication operations. The disadvantages of the shuffle-exchange network are the low degrees of regularity and modularity. Hence, this structure is not suitable for VLSI implementations. It has been shown that the network is not planar and cannot be embedded in silicon using an amount of area linearly proportional to the number of nodes [19].

3. Examples of VLSI Algorithms

The design of special-purpose VLSI chips to implement algorithms is illustrated by the implementation of a simple systolic priority queue [20]. A priority queue can be implemented by a linear array with constant response time for the insertion and extract minimum operations, independent of the number of elements in the priority queue, as opposed to $O(\log n)$ time, where n is the number of elements in the priority queue, as implemented by some balanced trees. Each cell in the array has two registers, A and B . The cell can access the register of its two neighbors as shown in Fig. 11. The A registers hold elements in the queue in sorted order with the smallest element in A , while the B registers hold elements that are being inserted into the queue. The elements of the queue are initialized to $+\infty$. The operations INSERT and EXTRACT MIN are performed at the left end of the priority queue. When elements are inserted into the priority queue, overflow elements become output at the right end. If the overflow element is $+\infty$, no overflow has occurred. If the overflow element is not $+\infty$, it is an indication of a real overflow.

The even- and odd-numbered cells pulsate alternately. At each pulse, the following operations are executed:

- (i) $B_i \leftarrow B_{i-1}$,
- (ii) arrange the elements in A_{i-1} , A_i , and B_i so that $A_{i-1} \leq A_i \leq B_i$.

The array pulses twice each time an operation is performed, once for the even-numbered cell and once for the odd-numbered cell. Cell 0 is a dummy cell that does not perform any operation. However, its registers can be altered by the host computer.

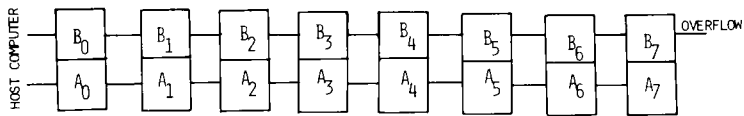


Fig. 11. A simple priority queue (from Leiserson [20]).

The operation $\text{INSERT}(Q,a)$ is implemented by placing the value $-\infty$ in A_0 and the item a in B_0 before cell 1 pulses. The element then travels to the right until it finds its proper place in the priority queue.

The EXTRACT MIN operation is implemented by loading A_0 and B_0 with the values $+\infty$; the minimum value can then be found in A_0 . The array is ready to execute a new operation after each pair of pulsations. The operations on the priority queue are shown in Fig. 12. The initial configuration in the figure has the insertion of items 9 and 15 already in progress. It is easily seen that only constant time is needed to execute the INSERTION and EXTRACT MIN operations. Because of the pipeline

	0	1	2	3	4	5	6	7	STEP
B	6		9		15		∞		0 INSERT(6)
A	$-\infty$	3	5	10	14	∞	∞	∞	
B		6		9		15		∞	1.1
A	$-\infty$	3	5	10	14	∞	∞	∞	
B		6		10		∞		∞	1.2
A	$-\infty$	3	5	9	14	15	∞	∞	
B			6		10		∞		2.1
A		3	5	9	14	15	∞	∞	
B	∞		6		14		∞		2.2 EXTRACT MIN
A	∞	3	5	9	10	15		∞	
B		∞		6		14		∞	3.1
A	$-\infty$	3	5	9	10	15	∞	∞	
B		∞		9		15		∞	3.2
A	3	$-\infty$	5	6	10	14	∞	∞	
B			∞		9		15		4.1
A		$-\infty$	5	6	10	14	∞	∞	
B	8		∞		10		∞		4.2 INSERT (8)
A	$-\infty$	5	∞	6	9	14	15	∞	

Fig. 12. Several steps in the execution of an array (from Leiserson [20]).

feature, no degradation results when many operations are requested in a row.

4. Design Methodology for Implementation of VLSI Algorithms

A methodology is essential to the design of any kind of complex system because it can decompose the complex and difficult problem into smaller subproblems that can be tackled more or less independently. The use of design methodology is especially important in the design of special-purpose VLSI chips because of the diversity of tasks and concerns in VLSI design [16,17,21,22]. It is important to decompose the design problem into small subtasks with boundaries between them that hide the implementation details of one from another.

The chip design problem can be decomposed in three ways: geometrically, functionally, and hierarchically. Geometrical decomposition can be obtained in geometrically regular algorithms from the spatial separation imposed between subsystems. The interior of one cell can thus be designed without concerning the interior details of another cell. The functional decomposition for complex cells can be provided by separating the circuit functions within each cell. The design of each functional block can then proceed more or less independently of the others. The idea of hierarchical decomposition is based on the existence of a hierarchy of abstract models of the chip, starting from the algorithm level, to the gate level, and then to the layout level. Each level of the hierarchy is concerned with an independent set of design issues, which serves as an implementation of the next higher level and a specification of the next lower level.

In order to arrive at the best division of tasks, all three types of decomposition must be performed consistently, that is, tasks decomposed geometrically must also be decomposed functionally as well as hierarchically. This decomposition can be facilitated by careful construction of a task-dependency graph before the beginning the design. The task-dependency graph contains all the subtasks to be performed, as well as the information needed for each and the precedence relations among them. An example of the task dependency of the design of a pattern-matching chip is shown in Fig. 13. It should be noticed that this dependency graph can be applied to the design of other chips of about the same scale. In the task-dependency graph, each subtask only considers one geometric region, one circuit function, and one level of the VLSI abstraction hierarchy, while the arrows indicate the flow of information between subtasks. With the aid of the task-dependency graph, the complex design process of special-purpose VLSI chip can be carried out systematically, one subtask at a time. Furthermore, many subtasks can be automated by computer-aided design systems. Hence, except for the algorithmic level, the design

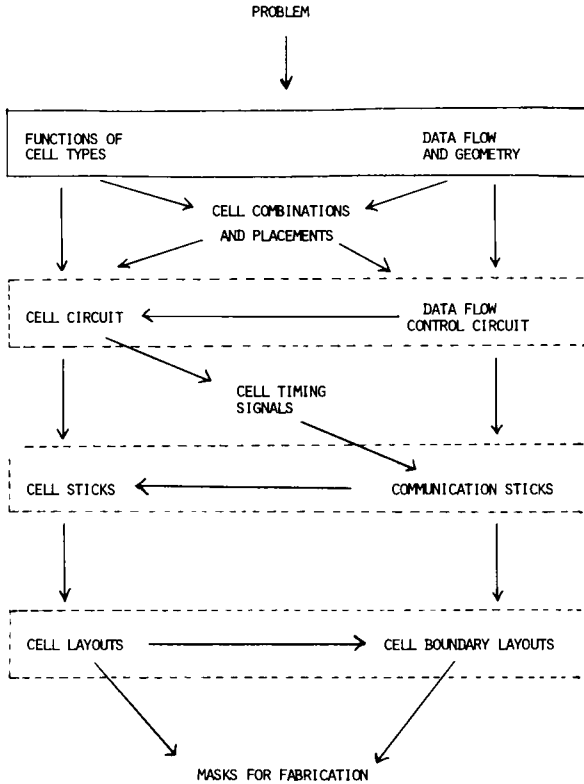


Fig. 13. Steps in the design of special-purpose VLSI chips (after Foster and Kung [16]).

process is relatively routine and straightforward. The algorithm design thus becomes the only difficult part in the design of special-purpose VLSI chip and requires substantial effort and experience. The preceding methodology can also be applied to the design of other VLSI chips.

B. Distributed Intelligence in System Components

Because of the large number of switching elements in the VLSI chip, it is possible to introduce more intelligence into different components of a computer system, such as memory and I/O processor, so that the processing load on the system can be distributed more evenly among the different modules of the system, at the same time reducing the the amount of communication among the different components. One example in this area is the design of a fast cellular associative memory, which expands the

functions of conventional associative memory [23]. The proposed associative memory can perform the following search operations: the basic equality search, the threshold searches (both greater than and less than searches), and the greatest value and the least value searches. Each of the search operations can be performed in one complete memory cycle using bit memory logic primarily. Based on the basic search schemes, an algorithm for ordered retrieval has been developed that outperforms all the other previous algorithms. The only limiting factor of this design is the large number of gates involved. Each bit cell in the associative memory requires 13 gates. In addition, there is extra logic associated with the registers and the controls. For a 32-bit word and 64-word memory, the design will need over 26,000 logic gates in the bit cells only, excluding all other register and control logics. However, this design become more feasible to implement with VLSI technology. Another extension to associative memory design, known as enhanced logic memory, is also aimed at the VLSI technology [24]. The design of more powerful I/O processors, such as the Intel 8089, has also been developed.

IV. CONCLUSION

The emergence of VLSI technology has opened many new architectural concepts that depart sharply from the design of conventional computer systems. The design of single-chip computers, special-purpose chips to implement algorithms, and more intelligent computer system modules, such as memory and I/O processors, are the basic impacts of VLSI technology on computer system design. Using the general-purpose or special-purpose chips as basic building blocks, very powerful but inexpensive computer systems can be designed. The low cost of these systems enables the design of special-purpose computer systems in which the architecture is tailored to the application [25]. However, the problem of complexity must be overcome in order to exploit the advantages of VLSI technology. Design methodology and automated design tools are needed far more urgently than ever in the generation of VLSI technology.

REFERENCES

1. S. P. Morse, W. B. Ravenel, S. Mazor, and W. B. Pohlman, Intel Microprocessors—8008 to 8086. *Computer* 42–60 (October 1980).
2. V. L. Rideout, Limits to improvement of silicon integrated circuits, *Proc. Compcom*, pp. 2–6 (1980).

3. F. Faggin, How VLSI impacts computer architecture, *IEEE Spectrum* (May 1978).
- 3a. G. Moore, VLSI: Some fundamental challenges, *IEEE Spectrum* (April 1979).
4. E. Bloch and D. Gtorage, Component progress: Its effect on high speed computer architecture and machine organization, in "High Speed Computer and Algorithm Organization" (D. Kuck, D. Lawrie, and A. Sameh, eds.). Academic Press, New York, 1977.
5. C. V. Freiman, VLSI high performance processors: How mixed the blessing? *Proc. Compcom* pp. 11–14 (1980).
6. C. Mead and C. Lynn, "Introduction to VLSI Systems." Addison-Wesley, Reading, Massachusetts, 1980.
7. D. A. Patterson and C. Sequin, Design considerations for the single-chip computer of the future. *IEEE Trans. Comput.* C-29, (1980).
8. H. G. Cragon, The elements of single-chip microcomputer architecture, *Computer* 27–41 (October 1980).
9. K. A. El-Yat, The Intel 8089: An integrated I/O processor, *Computer* 67–78 (June 1979).
10. J. D. Bryant and R. Longly, TMS9940 single chip microcomputer, *Proc. Electro* (1977).
11. E. Stritter and T. Gunter, A Microprocessor Architecture for a changing world: The Motorola 68000" *Computer* 43–52 (Feb. 1979).
12. A. H. Sequin, Single-chip computers, the new VLSI building blocks, *Proc. Caltech Conf. Very Large Scale Integration* pp. 435–446 (1979).
13. A. M. Despain and D. A. Patterson, X-tree: A tree structured multiprocessor computer architecture, *Proc. Symp. Comput. Architect.* 5th pp. 144–151 (1978).
14. D. A. Patterson, E. S. Fehr, and A. H. Sequin, Design considerations for the VLSI processor of X-Tree, *Proc. Ann. Symp. Comput. Architecture* pp. 90–100 (1979).
15. H. T. Kung, and C. E. Leiserson, Systolic Arrays (for VLSI), Technical Report, Carnegie-Mellon University Department of Computer Science (December, 1978).
16. M. J. Foster and H. T. Kung, The design of special-purpose VLSI chips, *Computer* 26–40 (January 1980).
17. M. J. Foster and H. T. Kung, Design of special-purpose VLSI chips: Example and opinions, *Proc. Ann. Symp. Computer. Architecture*, 7th pp. 300–307 (1980).
18. H. T. Kung, Let's design algorithms for VLSI systems, *Proc. Caltech Conf. Very Large Scale Integration* pp. 65–91 (1979).
19. C. D. Thompson, Area–time complexity for VLSI, *Ann. ACM Symp. Theory Comput.* (May 1979).
20. C. E. Leiserson, Systolic priority queues, *Proc. Caltech Conf. Very Large Scale Integration* pp. 199–214 (1979).
21. B. Lattin, VLSI design methodology: The problem of the 80's for microprocessor design, *Proc. Caltech Conf. Very Large Scale Integration* pp. 247–252 (1979).
22. W. M. VanCleemput, Hierarchical design for VLSI: Problems and advantages *Proc. Caltech Conf. Very Large Scale Integration* pp. 259–274 (1979).
23. C. V. Ramamoorthy, J. L. Turner, and B. W. Wah, A design of a fast cellular associative memory for ordered retrieval, *IEEE Trans. Comput.* C-27 800–815 (1978).
24. W. M. Denny, E. R. Buley, and E. Hatt, Logic-enhanced memories: An overview and some examples of their application to a radar tracking problem. *Proc. Caltech. Conf. Very Large Scale Integration* pp. 173–186 (1979).
25. J. L. Bentley and H. T. Kung, A tree machine for searching problems, *Proc. Int. Conf. Parallel Proc.* pp. 257–266 (1979).